

PHASE- 5:- Asynchronous JS, APIs ✓

✓ Asynchronous JS, Event Loop & Callbacks ✓

→ JS single thread synchronous prog —

JS Synchronous — ✓

→ Blocking Nature

→ flow → line 1, → line 2, line → 3



Call stack

Asynchronous →

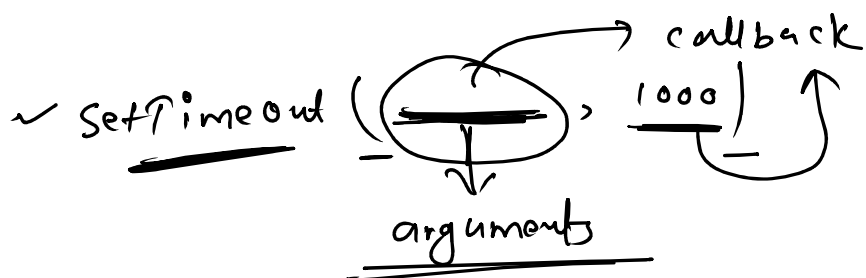
→ Some task take time for example —

→ API request
→ Timers
→ Reading file

} → database operation

callback

→ A callback is a function passed to another function to be executed later.



SetTimeout (`() => {}`
`console.log("Hello")`
`3, 1000`)

function run {

callback
→ SetTimeout (run, 2000)

`() => {}`
`log("Hello")`
`3`

JS itself mainly single-threaded

→ Browsers provides additional features to solve asynchronous task.

-1 Call Stack

→ Web API → Browsers → Macrotask (task queue)

→ Microtask → Promise (microtask queue)

→ Eventloop

Call stack - (Synchronous code)

↳ LIFO → Last in first out
↳ one function at a time.

Web APIs → Provided by Browser/Node.js
(Not JS engine)

Ex → handles things → timers, fetch,
DOM event etc.

↓ I/O →

task queue (macrotask queue)

↓
return → callback → Synchronous

↑
Call stack
pushed on

microtask queues → promise

→ handled by js engine

→ When you create a promise the engine allocates memory for the promise and tracks its resolution.

Priority →

Microtask > macrotask

Synchronous > Micro > Macro

Event Loop →

→ checks if call stack is empty

→ Runs all microtasks pushed call stack

↓
microtask first

→ Then moves one task (one macrotask) to call stack — —

→ check microtask =

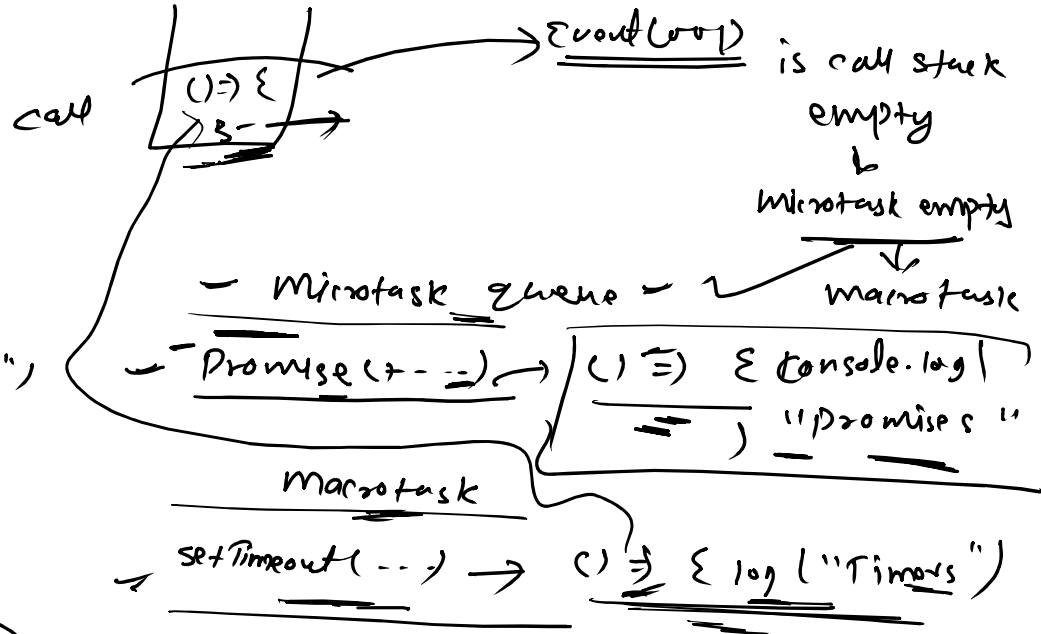
→ Repeat continuously

Ex →

```
console.log("start") -  
- setTimeout(() => {  
  console.log("Timer")  
  3, 1000)  
- Promise.resolve().then(() => {  
  console.log("promise solved"),  
  3)  
console.log("end")
```

Output

start
end
promise solved
Timer ✓



Execution order -

- ① Run synchronous code (call stack)
- ② moves async task to web API or, ss
- ③ when ready, add to microtask or, Task Queue
- ④ Event loop check

① Run all microtask first

② Then one task (one macro task)

③ Repeat

setTimeout (() => {})

← [promise.then(resolve(), then() => {})
log("Hello")]

3)
3, 1000)